Progression · Timeline · Graph

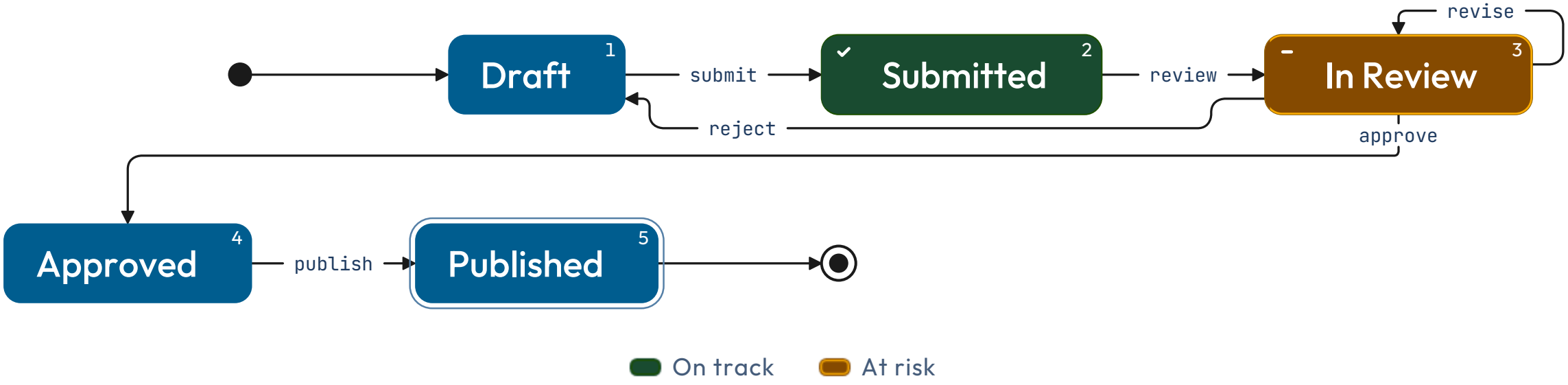
state-chart

Native state machine diagram — states as a list, transitions as arrows to a state's name, on the flowchart's grammar.

Submission lifecycle

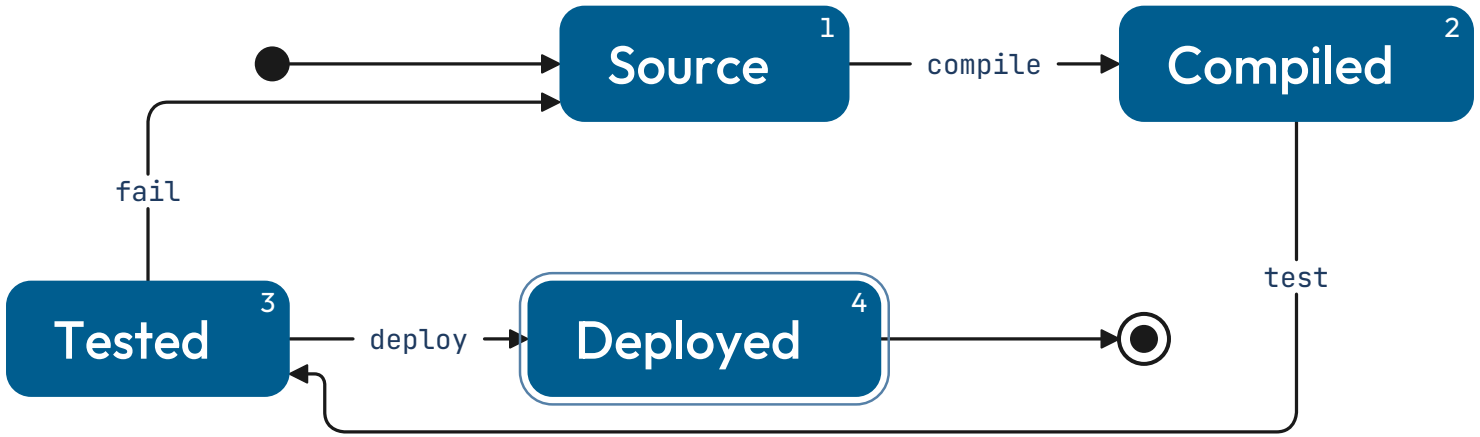
States connect; the arrows carry the rules.

How a draft moves from author to publication.

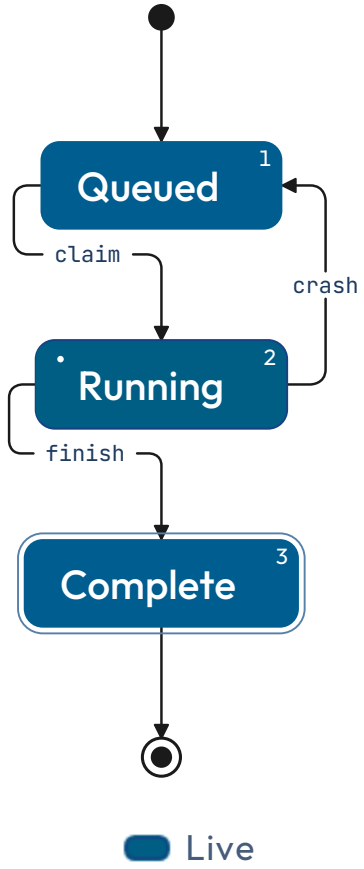


Rejected drafts return to the author; revisions stay in review.

lr flows the states left to right.



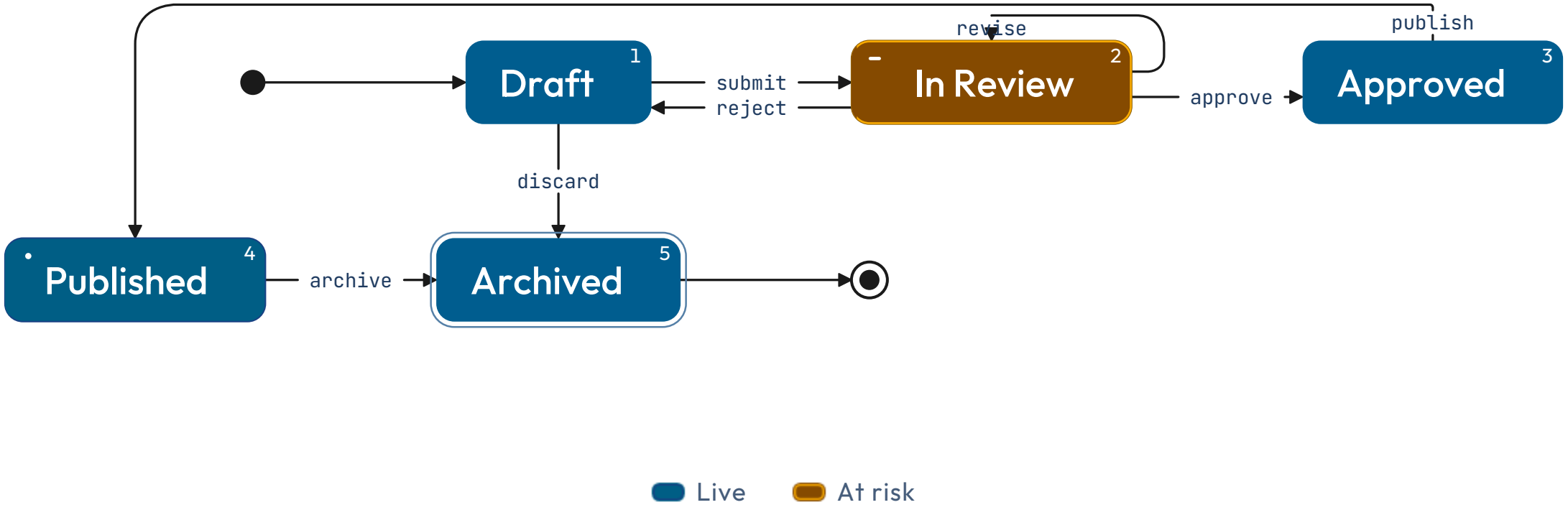
tb stacks the states top to bottom.



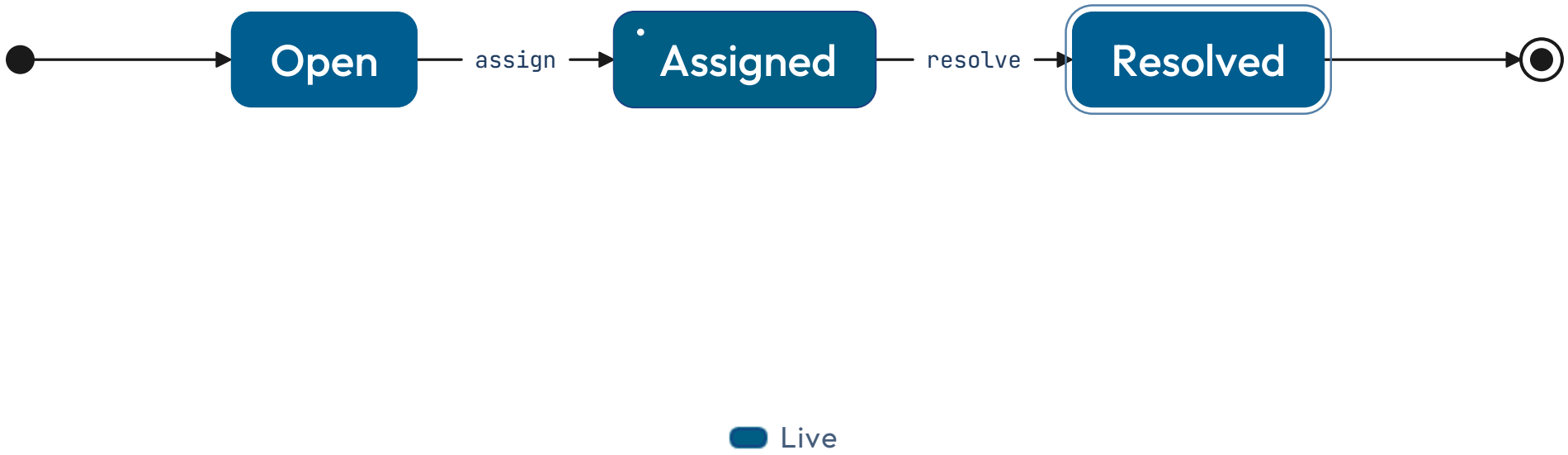
inline sets the chart beside its prose.



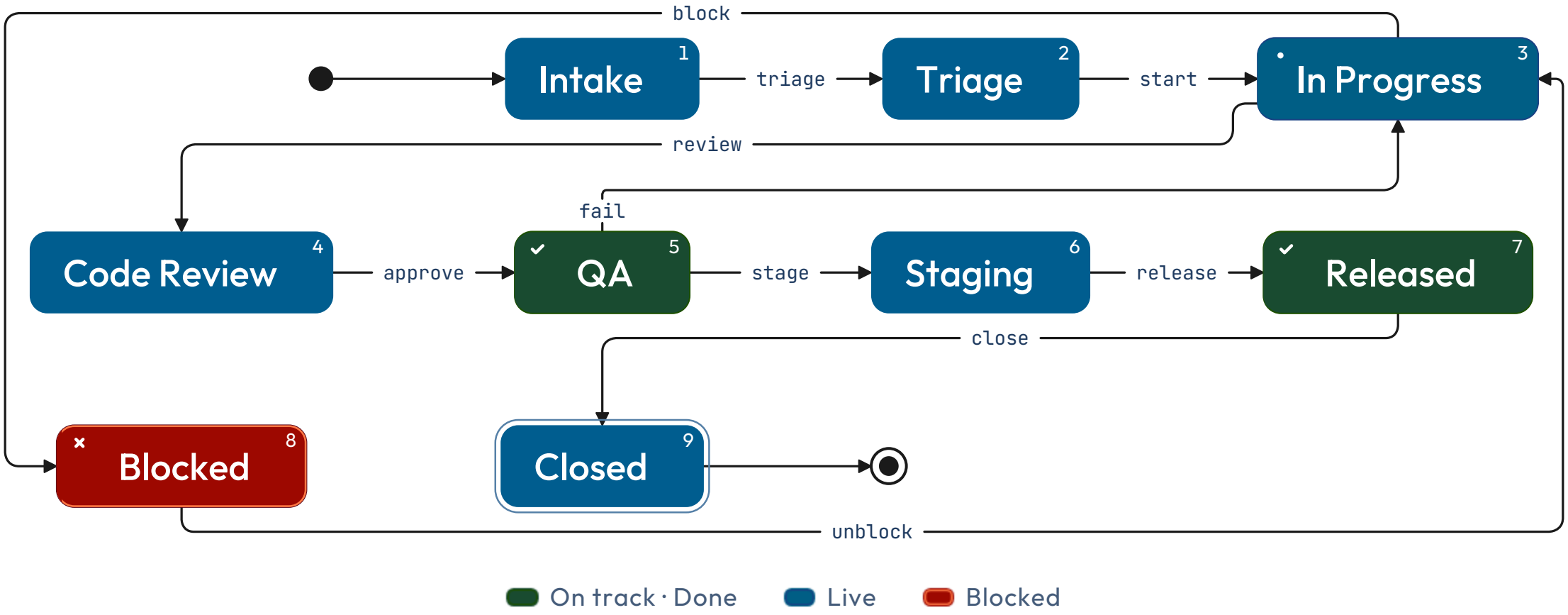
curved rounds the lines between states.



unnumbered drops the badges.

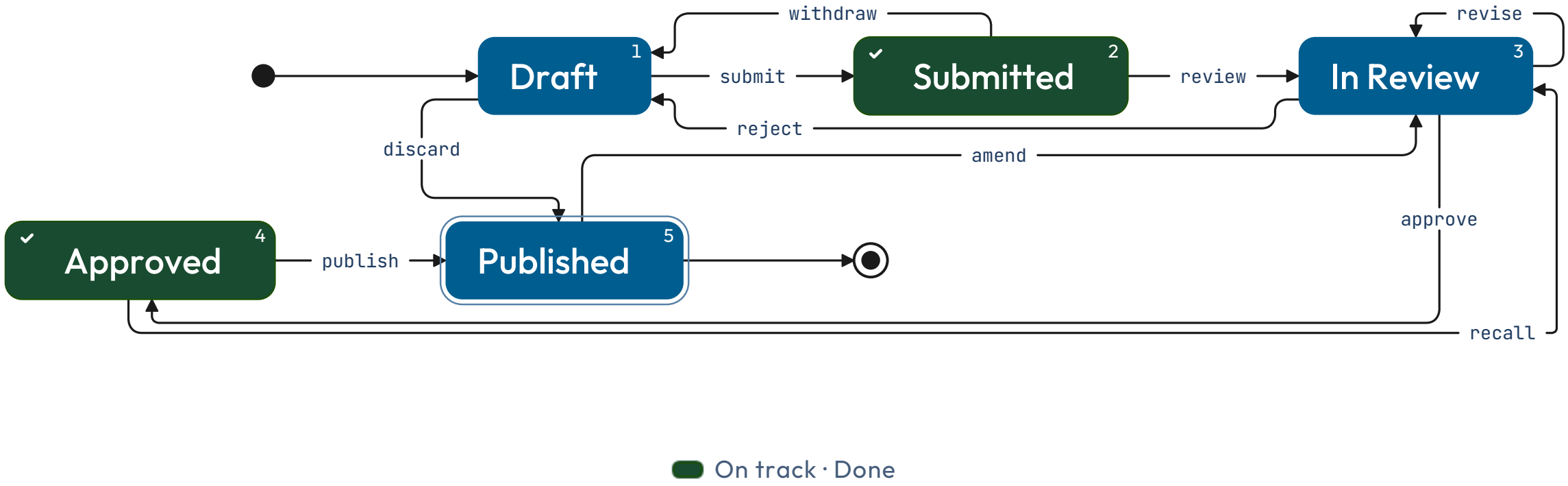


rearrange lets a side state sit beside the state it leaves.



Submission lifecycle

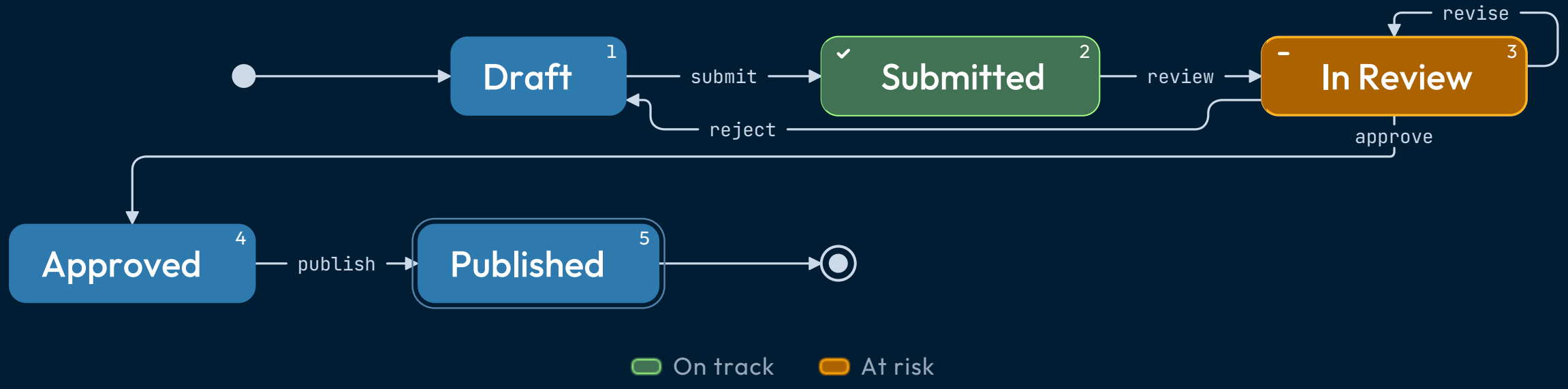
Stress test — back-edges, skips, self-loop.



Submission lifecycle

States connect; the arrows carry the rules.

How a draft moves from author to publication.

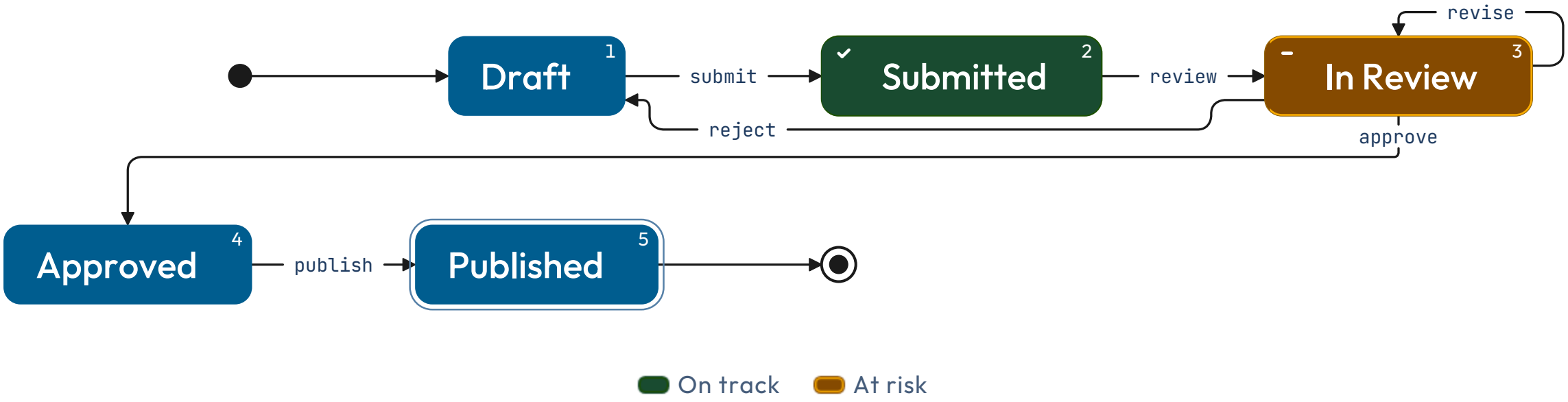


Rejected drafts return to the author; revisions stay in review.

Submission lifecycle

States connect; the arrows carry the rules.

How a draft moves from author to publication.

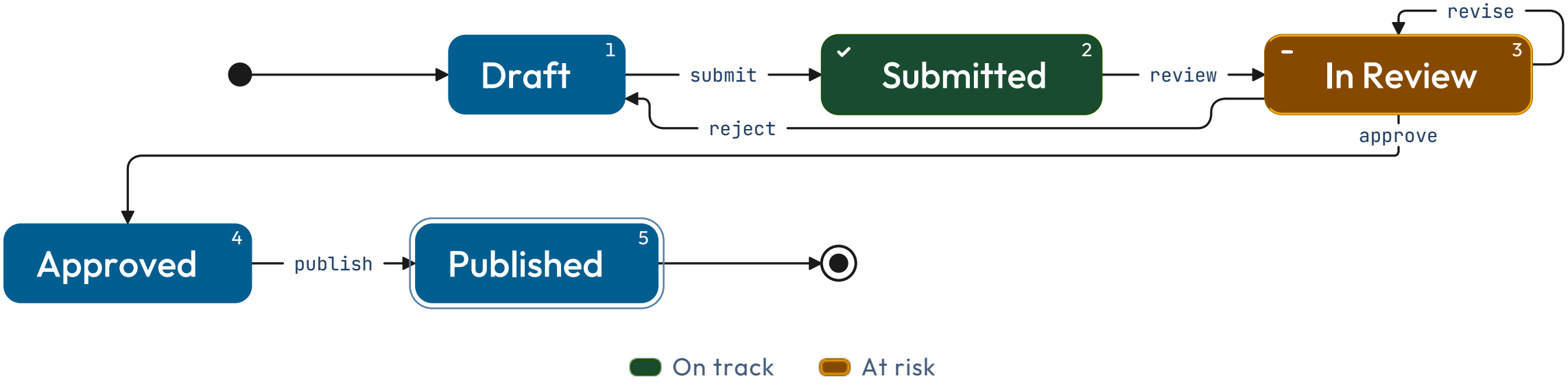


Rejected drafts return to the author; revisions stay in review.

Submission lifecycle

States connect; the arrows carry the rules.

How a draft moves from author to publication.



Rejected drafts return to the author; revisions stay in review.

When NOT to reach for state-chart.

More than ~12 states

Past about a dozen states the machine reads as a list. Group them into phases and show one phase at a time.

Parallel regions, history or guards

A composite state is a sub-list, but orthogonal regions, history states and guards are not in this grammar. Use a Mermaid fence via `diagram`.

Continuous processes

Stages that overlap or block (queue depth, throughput, capacity) read better as `gantt` or `kanban`. A state chart is for discrete, mutually exclusive states.

Related components

See also.

`diagram` — the machine has hierarchical states, parallel regions, or guards that need Mermaid's full state-diagram grammar

`journey` — the sequence is a user's path through tasks with mood / affect, not a system's discrete states

`timeline-list` — events are points in time rather than transitions between named states

`list-steps` — a linear procedure with no branching — state-chart is overkill if there are no choices to make

`roadmap` — parallel workstreams across phases, not a single machine's transitions